

Guía de Actividad 3 – GitHub ANÁLISIS Y DESARROLLO DE SOFTWARE

Thomas Isaza Chalarca

Instructor: Luis Fernando

Base datos SQL(Relacionales)

Institución: CTMA Sena

Antioquia

2025

**Guía de Actividad 3 – GitHub ANÁLISIS
Y DESARROLLO DE SOFTWARE**
CENTRO DE TECNOLOGÍA DE LA MANUFACTURA AVANZADA MEDELLÍN



PROCESO DIRECCIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE

Palabras claves (key words)

Git, GitHub Control de versiones, Repositorio, Commit, Branch (Rama), Merge (Fusionar), Pull request, Fork (Bifurcación), GitHub Actions, Push, Clone (Clonar).

Preguntas de Introducción:

- ¿Qué es un repositorio en Git y cuál es su función dentro de un proyecto de desarrollo?
Un repositorio en Git es el componente más básico donde se almacena el código, los archivos y el historial de revisiones de cada archivo de un proyecto. Su función Permite administrar, debatir y mantener el trabajo del equipo, pudiendo ser público o privado y contar con múltiples colaboradores
- ¿Qué problema resuelve el uso de sistemas de control de versiones como Git en proyectos de desarrollo de software?
Los sistemas de control de versiones como Git resuelven el problema de gestionar los cambios en los archivos a lo largo del tiempo, permitiendo recuperar versiones anteriores, comparar cambios, identificar quién y cuándo hizo modificaciones, y restaurar archivos perdidos o dañados. Esto es fundamental para la colaboración y la integridad del proyecto
- ¿Cómo permite Git la colaboración entre varios desarrolladores sin que sus cambios interfieran entre sí?
Git permite que cada desarrollador trabaje en su propia copia local del repositorio y en ramas (**branches**) separadas. Así, pueden desarrollar nuevas funciones o corregir errores de manera independiente, sin afectar el código principal ni el trabajo de otros. Los cambios se integran al repositorio central solo cuando están listos, minimizando interferencias

- ¿Cuál es la diferencia entre un commit y un push en el flujo de trabajo de Git?
Commit: Guarda los cambios realizados en el repositorio local, registrando un punto en la historia del proyecto.
Push: Envía esos cambios confirmados (commits) desde el repositorio local al repositorio remoto, haciéndolos accesibles a otros colaboradores
- ¿Qué es un pull request en GitHub y cómo ayuda a revisar y fusionar los cambios de los colaboradores?
 Un pull request (**PR**) es una solicitud para fusionar cambios desde una rama (generalmente de un colaborador) a otra rama (como la principal) en un repositorio. Permite que otros miembros del equipo revisen, discutan y aprueben los cambios antes de integrarlos, asegurando así la calidad y consistencia del código.
- ¿Qué ventajas ofrece el uso de ramas (branches) en Git cuando se está desarrollando una nueva funcionalidad o corrigiendo un error?
 El uso de **ramas** permite:
 1. Desarrollar nuevas funcionalidades o corregir errores en un entorno aislado, sin afectar la rama principal.
 2. Facilitar la colaboración y minimizar conflictos.
 3. Probar y revisar los cambios antes de fusionarlos al código principal, manteniendo la estabilidad del proyecto
- ¿En qué situaciones se recomienda usar un fork en GitHub?
 Se recomienda usar un **fork** cuando se desea contribuir a un proyecto del que no se tiene acceso directo de escritura, como ocurre en proyectos de código abierto. El fork crea una copia completa del repositorio en la cuenta personal del usuario, permitiendo trabajar libremente y luego proponer cambios al proyecto original mediante pull request.

Actividad practica:

Usted está trabajando en el desarrollo de una plataforma web para la empresa “Coquito Amarillo S.A.S.”, cuyo objetivo es evaluar el nivel de satisfacción personal de sus empleados a través de la “Rueda de la vida” (un enfoque utilizado en psicología para medir el bienestar en diferentes áreas). La plataforma permite a los empleados completar un cuestionario interactivo, visualizar los resultados y generar un informe con recomendaciones personalizadas.

Durante el proceso de desarrollo, el equipo se enfrenta a varios desafíos relacionados con el control de versiones y la colaboración efectiva. El proyecto está siendo desarrollado por un equipo de varios miembros, y la coordinación de los cambios, la integración de nuevas funcionalidades y el despliegue del sitio web son aspectos críticos.

- ¿Cómo podría estructurar el flujo de trabajo del equipo utilizando Git y GitHub para asegurar que todos los miembros colaboren de manera eficiente y no se presenten conflictos de código?

La mejor práctica es implementar un **flujo de trabajo basado en ramas**. Cada desarrollador debe crear una rama específica para cada nueva funcionalidad o corrección, partiendo de la rama principal (por ejemplo, main o develop). Así, cada miembro trabaja de manera aislada y puede realizar commits frecuentes en su propia rama, sin afectar el código estable del proyecto. Una vez completada la tarea, se debe abrir un **pull request** para que el equipo revise los cambios antes de fusionarlos a la rama principal. Esto permite detectar errores, discutir mejoras y mantener la integridad del código. Además, es recomendable mantener ramas separadas para desarrollo y producción, y usar ramas de tipo “feature”, “bugfix” o “hotfix” según la naturaleza del trabajo

- Imagine que un miembro del equipo ha realizado cambios importantes en una funcionalidad de la evaluación de la rueda de la vida, pero al hacer un Merge de la rama en la que estaba trabajando, se presentan conflictos con la rama principal del proyecto. ¿Cómo resolvería este conflicto utilizando Git y GitHub?

Cuando al intentar fusionar una rama con la rama principal surgen conflictos, Git marca los archivos afectados y detiene el proceso de fusión. Para resolverlo:

- Se identifican los archivos en conflicto (Git los mostrará como “Unmerged”).
- Se abren los archivos en el editor de código; y se verán las secciones conflictivas delimitadas por <<<<<<, ===== y >>>>>>.
- Se elige manualmente qué cambios conservar, combinando o adaptando el código según sea necesario.
- Se Guardan los archivos y se usa git add <archivo> para marcar los conflictos como resueltos.
- Acto siguiente se realiza un nuevo commit (git commit) para completar la fusión.
- Si el conflicto es muy complejo o fue un error intentar fusionar, se puede abortar el proceso con git merge --abort

- Como parte de su rol en el proyecto, decide automatizar las pruebas unitarias de las funcionalidades de la web utilizando GitHub Actions. ¿Qué pasos seguiría para configurar un flujo de trabajo en GitHub Actions que ejecute las pruebas automáticamente con cada push a la rama principal y despliegue el sitio web de manera continua?

Para automatizar pruebas unitarias y el despliegue continuo con **GitHub Actions**:

- Crear un archivo de flujo de trabajo (por ejemplo, `.github/workflows/ci-cd.yml`) en el repositorio.
- Definir los eventos que activarán el flujo, como push o pull request a la rama principal.
- Agregar un job para instalar dependencias y ejecutar pruebas unitarias (por ejemplo, usando Node.js, Python, etc.).
- Si las pruebas pasan, se define un paso para desplegar el sitio web (usando scripts personalizados, servicios como GitHub Pages, o integraciones con proveedores externos).
- Por último, se podría agregar ambientes y controles para requerir aprobaciones antes de desplegar a producción

Un ejemplo básico de configuración en YAML sería:

name: CI/CD

on:

push:

branches: [main]

jobs:

build-and-test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- name: Instalar dependencias
- run: npm install
- name: Ejecutar pruebas
- run: npm test

deploy:

needs: build-and-test

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- name: Desplegar sitio
- run: ./deploy.sh

Este flujo asegura que cada cambio en la rama principal sea probado y desplegado automáticamente, incrementando la calidad y la velocidad del desarrollo

Actividad 1: Configuración Inicial

Para aprender debemos hacer, empecemos por realizar la configuración del repositorio y la estructura del proyecto en GitHub; para ello vamos a crear nuestro perfil de desarrollador y un repositorio en GitHub y configuraremos su estructura básica:

1. Crear su perfil de usuario en la plataforma GitHub (<https://www.github.com>).
2. Crear en su perfil una organización (Ej.: Coquito Amarillo S.A.S)
3. Crear en su organización un proyecto (Ej.: Web Corporativa)
4. Crear en su proyecto un repositorio con un nombre apropiado, como “coquito amarillo-web”.
5. Clonar el repositorio en su máquina local utilizando ``git clone <URL del repositorio>``.
6. Crear la estructura inicial de directorios dentro del repositorio (por ejemplo, una carpeta ``documentación`` para la gestión administrativa del proyecto, ``src`` para los archivos del código y una carpeta ``assets`` para imágenes, estilos y otros recursos).
7. Realizar el primer *commit* para guardar la estructura general y los archivos base del proyecto (*Readme*, *Licenciamiento*, *Requerimientos*, *product backlog*, *HTML*, *CSS*, imágenes).
8. Subir los cambios al repositorio remoto con el comando ``git push``. Herramientas de Git a utilizar:
 - ✓ ``git init``
 - ✓ ``git clone``
 - ✓ ``git add``
 - ✓ ``git commit``
 - ✓ ``git push``

Evidencias: Sustentación de su proyecto, presentando su perfil de GitHub y la estructura general de la organización, proyecto y repositorio.

Actividad 2: Ramas

Una de las principales características de GitHub es la posibilidad de usar ramas para mantener aislados los aspectos del proyecto y asegurar una mayor agilidad en su desarrollo. Utilizar ramas para desarrollar funcionalidades específicas sin afectar la versión principal del proyecto permitirá tener mayor control de los flujos de trabajo y hacer un seguimiento más adecuado a los colaboradores y sus asignaciones.

Ejecute en su proyecto las siguientes actividades:

1. Crear una nueva rama para una funcionalidad específica, como el diseño de la página principal o la implementación de la evaluación de la rueda de la vida.
 - a. Ejemplo: **``git checkout -b diseño-página-inicial``**
2. Trabajar en los archivos correspondientes (HTML, CSS, python) dentro de esa rama.
3. Realizar *commits* frecuentemente para registrar los avances realizados en la rama.
4. Una vez completada la funcionalidad, realizar un *merge* de la rama con la rama principal (*main*) mediante un *pull request* en GitHub.
5. Resolver cualquier conflicto que pueda surgir al intentar fusionar las ramas.

Herramientas de Git a utilizar:

- ✓ **``git branch``**
- ✓ **``git checkout``**
- ✓ **``git merge``**
- ✓ ***Pull Request en GitHub***

Evidencias:

- ✓ Sustentación de su proyecto, presentando su proyecto en GitHub y la estructura general del repositorio.

Actividad 3: Pull Request e Issues

Nadie trabaja solo, por eso es esencial en el desarrollo de software la capacidad de colaborar, pero la colaboración tiene sus costos. Fomentar la colaboración utilizando **pull request** e **issues** para gestionar tareas y problemas facilita la construcción de proyectos de mayor tamaño y complejidad.

Realice las siguientes actividades para comprender mejor estos aspectos:

1. Crear un **issue** en GitHub para cualquier tarea específica, como "Implementar la evaluación de la rueda de la vida para empleados".
2. Asignar el **issue** a un miembro del equipo o a usted mismo.
3. Desarrollar la funcionalidad necesaria en una nueva rama (por ejemplo, `evaluaciónrueda-vida`).
4. Una vez completado, abrir un **pull request** para fusionar los cambios en la rama principal y mencionar el **issue** relacionado en el **Pull Request**.
5. Revisar y discutir el **pull request** en el equipo, realizar pruebas y hacer ajustes si es necesario.
6. Una vez aprobado, fusionar el **pull request** y cerrar el **issue**.

Herramientas de Git y GitHub a utilizar:

- ✓ **`git branch`**
- ✓ ***Pull Request en GitHub***
- ✓ ***Issues en GitHub***

Evidencias:

- ✓ Sustentación de su proyecto, presentando su repositorio en GitHub y el historial de actividades relacionado.

NO FUNCIONO GIT HUB. (ERROR CON SUS FUNCIONES, no encontré manera de solucionarlo)

Actividad 4: Actions

Automatización de Flujos de Trabajo con GitHub Actions permite por ejemplo automatizar tareas como la ejecución de pruebas unitarias o el despliegue de la web al entorno de producción.

A continuación se propone un reto para que el aprendiz intente desarrollar este tipo de actividad en el proyecto:

1. Configurar un archivo de flujo de trabajo en **GitHub Actions** para automatizar el proceso de CI/CD (Integración continua y despliegue continuo).
2. Crear un archivo `.yaml` en el directorio `.github/workflows` para definir el flujo de trabajo, por ejemplo, para ejecutar pruebas unitarias sobre el código cada vez que se haga un **push** a la rama principal.
3. Implementar pasos en el flujo de trabajo que automáticamente desplieguen la web en un entorno de producción o *staging* (como *GitHub Pages*, *Netlify* o *Heroku*) cuando se realice un **push**.
4. Configurar el flujo de trabajo para ejecutar **pruebas de integración** y revisar el código en cada **pull request**, para asegurarse de que no haya errores.
5. Verificar el funcionamiento de **GitHub Actions** observando los resultados en la pestaña de **Actions** en GitHub.

Herramientas de GitHub a utilizar:

- ✓ GitHub Actions
- ✓ Archivos de configuración `.yaml`

Evidencias:

- ✓ Sustentación de su proyecto, presentando su repositorio en GitHub y el historial de actividades relacionado.

GLOSARIO

- Git: Sistema de control de versiones distribuido utilizado para gestionar y realizar un seguimiento de los cambios en el código fuente durante el desarrollo de software. Permite a los desarrolladores trabajar de forma independiente en diferentes partes de un proyecto.
- Repositorio (Repository): Es un espacio donde se almacenan todos los archivos y el historial de cambios de un proyecto. Puede ser local (en tu máquina) o remoto (en un servidor como GitHub).
- Commit: Acción de guardar los cambios en el repositorio. Un "commit" registra un conjunto de modificaciones, y cada uno tiene un mensaje que describe lo que se ha cambiado. Cada commit tiene un identificador único (hash).
- Branch (Rama): Una rama es una versión paralela del proyecto. Permite trabajar en nuevas funcionalidades sin alterar el código principal. La rama principal por lo general se llama `main` o `master`.
- Merge (Fusionar): Acción de combinar los cambios de una rama con otra. Generalmente, se fusionan cambios de una rama secundaria (por ejemplo, una rama de características) a la rama principal.
- Clone (Clonar): Acción de crear una copia local de un repositorio remoto. Se utiliza para obtener una versión del proyecto desde un repositorio en línea.
- Pull: Es el comando para obtener los cambios más recientes de un repositorio remoto y fusionarlos con el repositorio local.
- Push: Es el comando utilizado para enviar los cambios realizados en el repositorio local al repositorio remoto.
- Staging Area (Área de preparación): Es el área donde se colocan los cambios que serán parte del siguiente commit. Permite a los desarrolladores seleccionar qué cambios se van a guardar.
- Diff (Diferencias): Muestra las diferencias entre dos versiones de un archivo o entre dos commits. Esto ayuda a revisar los cambios antes de hacer un commit.
- HEAD: Es una referencia a la rama o commit actual. HEAD apunta al último commit realizado en la rama en la que se está trabajando.
- Fork (Bifurcación): Crear una copia de un repositorio para realizar cambios. Es común en proyectos de código abierto, donde los desarrolladores crean un fork del repositorio para trabajar de manera independiente.
- Rebase: Permite incorporar cambios de una rama en otra de manera más lineal, reescribiendo el historial de commits.
- Tag (Etiqueta): Es una referencia fija a un punto específico en la historia de un repositorio, generalmente usada para marcar versiones o lanzamientos importantes.

- Conflict (Conflicto): Ocurre cuando dos ramas modifican el mismo fragmento de un archivo de manera diferente. Git no puede fusionar estos cambios automáticamente y requiere intervención manual.
- GitHub: Plataforma web que utiliza Git para ofrecer almacenamiento de repositorios de código, gestión de versiones, y colaboración en proyectos. GitHub permite a los desarrolladores almacenar, gestionar y compartir su código de manera eficiente.
- Pull Request (PR): Es una solicitud para fusionar los cambios realizados en una rama de un repositorio a otra rama (generalmente a la rama principal). Es una herramienta clave para la colaboración en proyectos de desarrollo.
- Issue: Herramienta de GitHub para gestionar y rastrear errores, mejoras y tareas relacionadas con un proyecto. Los desarrolladores pueden usar Issues para discutir cambios y mejoras.
- Actions: Herramienta de GitHub que permite automatizar flujos de trabajo, como compilaciones, pruebas y despliegues, directamente desde el repositorio de GitHub.
- Wiki: Espacio en GitHub donde los proyectos pueden almacenar documentación, guías, manuales y otros recursos relacionados con el proyecto.
- GitHub Pages: Funcionalidad de GitHub que permite alojar sitios web estáticos directamente desde un repositorio de GitHub. Es útil para proyectos de documentación o sitios web personales.
- Gist: Servicio de GitHub que permite compartir fragmentos de código o notas de manera sencilla. Un gist es un archivo individual o un pequeño conjunto de archivos.
- Collaborators (Colaboradores): Personas que tienen acceso a un repositorio privado para realizar cambios, enviar pull requests o simplemente leer el código.
- Stars: Son "me gusta" o favoritos que los usuarios de GitHub pueden dar a un repositorio para indicar su interés o apoyo al proyecto.
- Contributors (Contribuyentes): Son los usuarios que han realizado contribuciones a un repositorio, ya sea a través de commits, pull requests, o reportes de problemas.
- Actions Workflow (Flujo de trabajo de GitHub Actions): Definición de los procesos automáticos (como CI/CD) que GitHub ejecuta en respuesta a ciertos eventos, como cuando se hace un commit o un pull request.
- Commit History (Historial de commits): Registro de todos los commits realizados en un repositorio. Permite ver qué cambios se hicieron, cuándo y por quién.
- Webhooks: Son mecanismos que permiten a GitHub notificar a un sistema externo sobre eventos específicos de un repositorio, como un push o pull request.

ACTIVIDADES DE EVALUACIÓN

La evaluación de los aprendizajes será realizada en base a los siguientes criterios:

- ✓ Calidad en el uso de GitHub: ¿Se exploraron nuevos elementos adicionales a los mostrados en el documento? ¿Está bien estructurado el esquema organizaciónproyecto-repositorio?
- ✓ ¿Se identificaron y apropiaron herramientas adicionales que faciliten la creación del código?
- ✓ Calidad de la documentación: ¿La documentación de los resultados es clara, precisa y contiene todos los detalles necesarios?

Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
---------------------------	-------------------------	---------------------------------------

Evidencias de Conocimiento: Prueba de conocimiento sobre principios y conceptos de Git y GitHub (Cuestionario).	✓ Diseña la estructura del repositorio de acuerdo con los requerimientos especificados. ✓ Define el proyecto y sus características de acuerdo con las condiciones del entorno de producción.	Observación directa, entrevistas, pruebas escritas, proyecto practica de aula, informe final.
Evidencias de Desempeño: Verificación de los conceptos aplicados al diseño de la organización, proyecto, repositorio. (Listas de Chequeo, sustentación).	✓ Realiza los entrenamientos propuestos por el instructor. ✓ Documenta las actividades para mantener la trazabilidad del proyecto en GitHub.	
Evidencias de Producto: Entrega del repositorio en línea (Sustentación, Listas de Chequeo).		

EVIDENCIAS A PRESENTAR

Evidencias por desempeño

- ✓ *Desarrollo completo de las actividades planteadas en el documento.*

Evidencias de conocimiento

- ✓ *Informe en formato PDF con las respuestas a cada una de las preguntas y retos planteados en el documento.*

Evidencias de producto

- ✓ *Perfil de GitHub, con su respectiva organización, proyecto y repositorio creados.*

- ✓ *Informe en formato PDF con las evidencias gráficas de los procesos ejecutados para cumplir con las actividades y sus correspondientes observaciones, descripciones y conclusiones.*

REFERENTES BIBLIOGRÁFICOS

- ✓ Chacon, S., & Straub, B. (2014). Pro Git (2ª ed.). Apress.
- ✓ GitHub, Inc. (2020). Git Pocket Guide. O'Reilly Media.
- ✓ Bell, J. (2021). Learning GitHub Actions. Packt Publishing.
- ✓ Loeliger, J., & McCullough, M. (2021). Version Control with Git (3ª ed.). O'Reilly Media.

Cibergrafía

- ✓ itHub. (s.f.). GitHub Docs: Introducción a GitHub. Recuperado el 6 de marzo de 2025, de <https://docs.github.com/es/github>
- ✓ Microsoft. (s.f.). Introducción a Git y GitHub en Visual Studio Code. Recuperado el 6 de marzo de 2025, de <https://learn.microsoft.com/es-es/github>

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	Luis Fernando Sánchez Pérez.	Instructor	CTMA/TICS-Electrónica.	20/4/2025

CONTROL DE CAMBIOS (diligenciar únicamente si realiza ajustes a la guía)

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)	Luis Fernando Sánchez Pérez	Instructor	CTMA/TICS-Electrónica.	9/5/2025	Modificación de Actividades de Apropiación del conocimiento